In [1]:
```
# ==============================
# Lab 2: Shallow vs Deep Networks
# ==============================
```

In [2]:
```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.datasets import make_classification

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

In [3]:
```
# ------------------------------
# 1. Generate Random Data
# ------------------------------

X, y = make_classification(
    n_samples=1000,
    n_features=3,
    n_informative=3,
    n_redundant=0,
    n_classes=2,
    random_state=42
)
```

In [4]:
```
# ------------------------------
# 2. Preprocess Data
# ------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

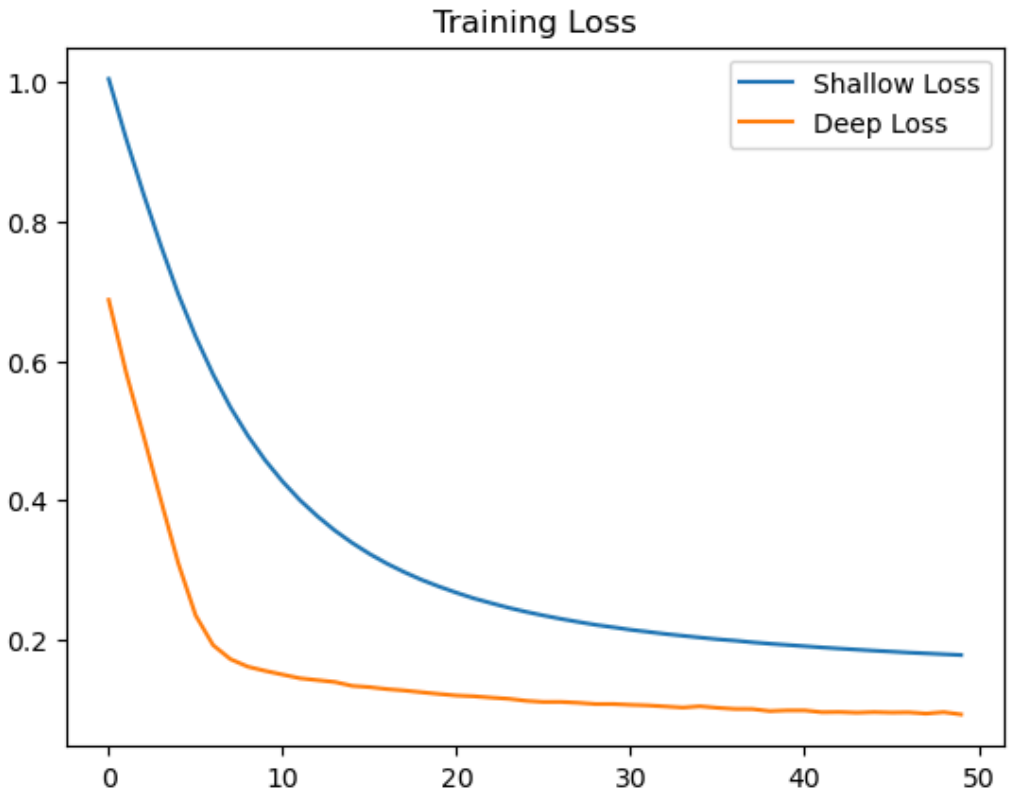
In [5]:
```
# ------------------------------
# 3. Shallow Neural Network
# ------------------------------

shallow_model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(3,)),
    layers.Dense(1, activation='sigmoid')
])

shallow_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_shallow = shallow_model.fit(
    X_train, y_train,
    epochs=50,
    validation_split=0.2,
    verbose=0
)
```

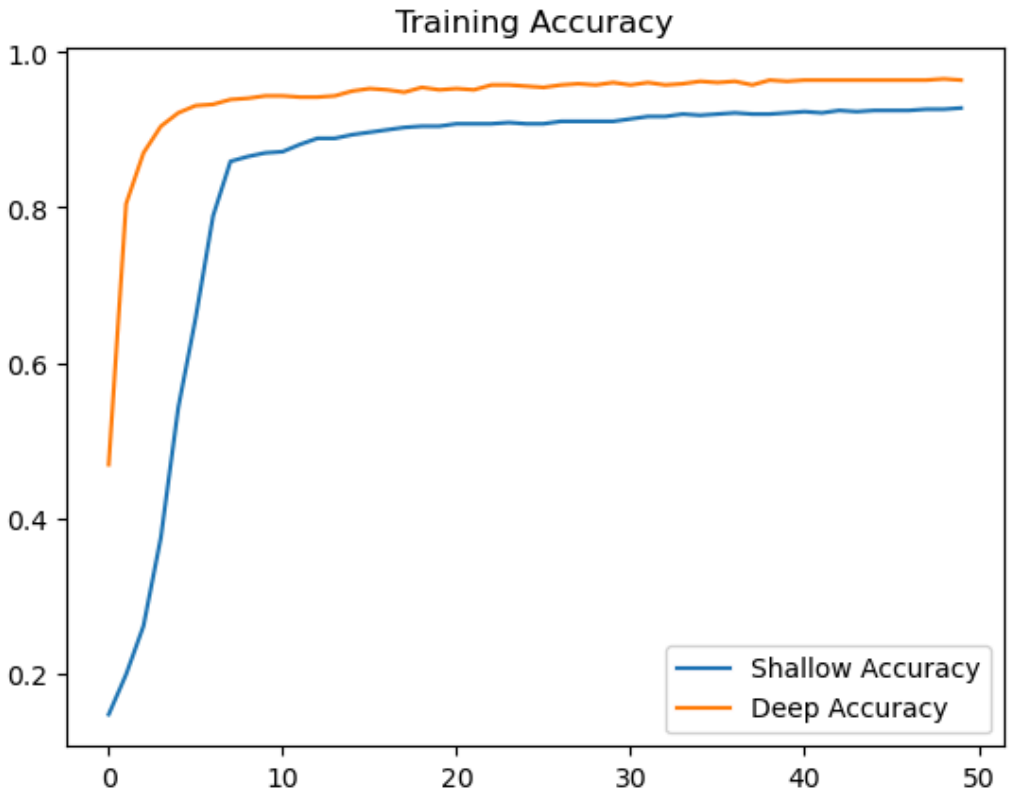
In [6]:
```
# ------------------------------
# 4. Deep Neural Network
# ------------------------------

deep_model = keras.Sequential([
    layers.Dense(32, activation='relu', input_shape=(3,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

deep_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history_deep = deep_model.fit(
    X_train, y_train,
    epochs=50,
    validation_split=0.2,
    verbose=0
)
```

In [7]:
```
# ------------------------------
# 5. Evaluation
# ------------------------------

def evaluate_model(model, X_test, y_test, name):
    y_pred_prob = model.predict(X_test)
    y_pred = (y_pred_prob > 0.5).astype(int)

    print(f"\n{name} Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall:", recall_score(y_test, y_pred))
    print("F1 Score:", f1_score(y_test, y_pred))


evaluate_model(shallow_model, X_test, y_test, "Shallow Model")
evaluate_model(deep_model, X_test, y_test, "Deep Model")
```

```
7/7 [==============================] - 0s 2ms/step

Shallow Model Performance:
Accuracy: 0.91
Precision: 0.9
Recall: 0.9
F1 Score: 0.9
7/7 [==============================] - 0s 2ms/step

Deep Model Performance:
Accuracy: 0.935
Precision: 0.9052631578947369
Recall: 0.9555555555555556
F1 Score: 0.9297297297297298
```

In [8]:
```
# ------------------------------
# 6. Plot Training Curves
# ------------------------------

plt.figure()
plt.plot(history_shallow.history['loss'], label='Shallow Loss')
plt.plot(history_deep.history['loss'], label='Deep Loss')
plt.title("Training Loss")
plt.legend()
plt.show()

plt.figure()
plt.plot(history_shallow.history['accuracy'], label='Shallow Accuracy')
plt.plot(history_deep.history['accuracy'], label='Deep Accuracy')
plt.title("Training Accuracy")
plt.legend()
plt.show()
```

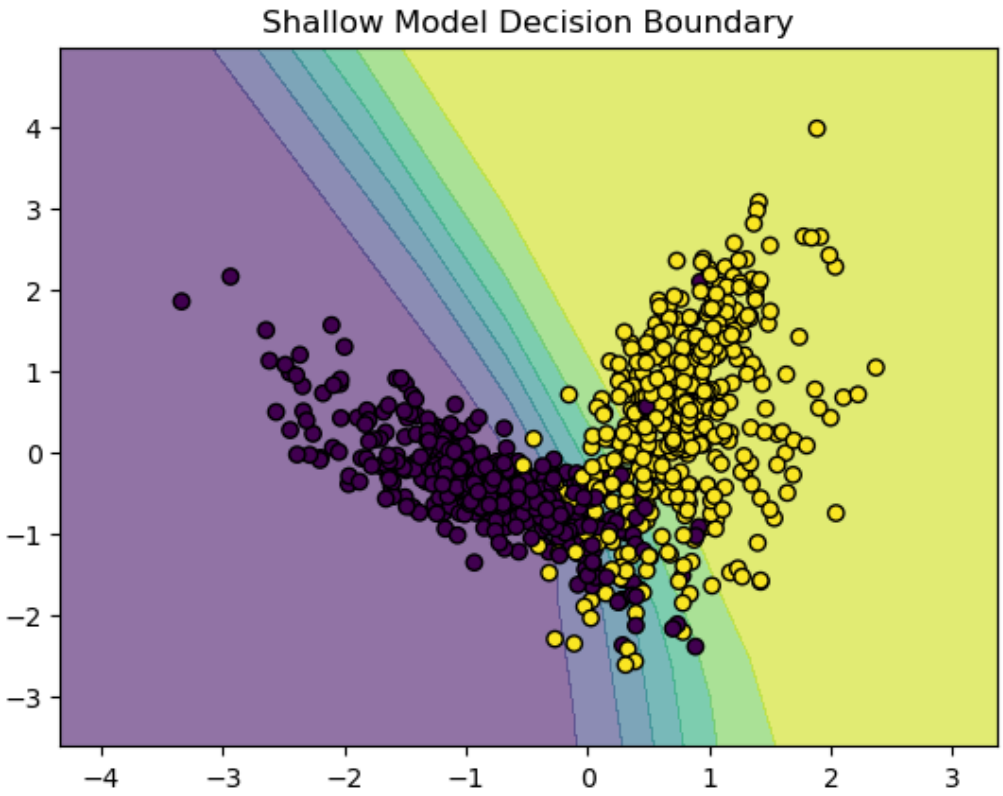




In [9]:
```
# ------------------------------
# 7. Decision Boundary Plot
# (Using first 2 features only for visualization)
# ------------------------------

def plot_decision_boundary(model, X, y, title):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )

    grid = np.c_[xx.ravel(), yy.ravel(), np.zeros(len(xx.ravel()))]
    grid_scaled = scaler.transform(grid)

    Z = model.predict(grid_scaled)
    Z = Z.reshape(xx.shape)

    plt.figure()
    plt.contourf(xx, yy, Z, alpha=0.6)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k')
    plt.title(title)
    plt.show()
```

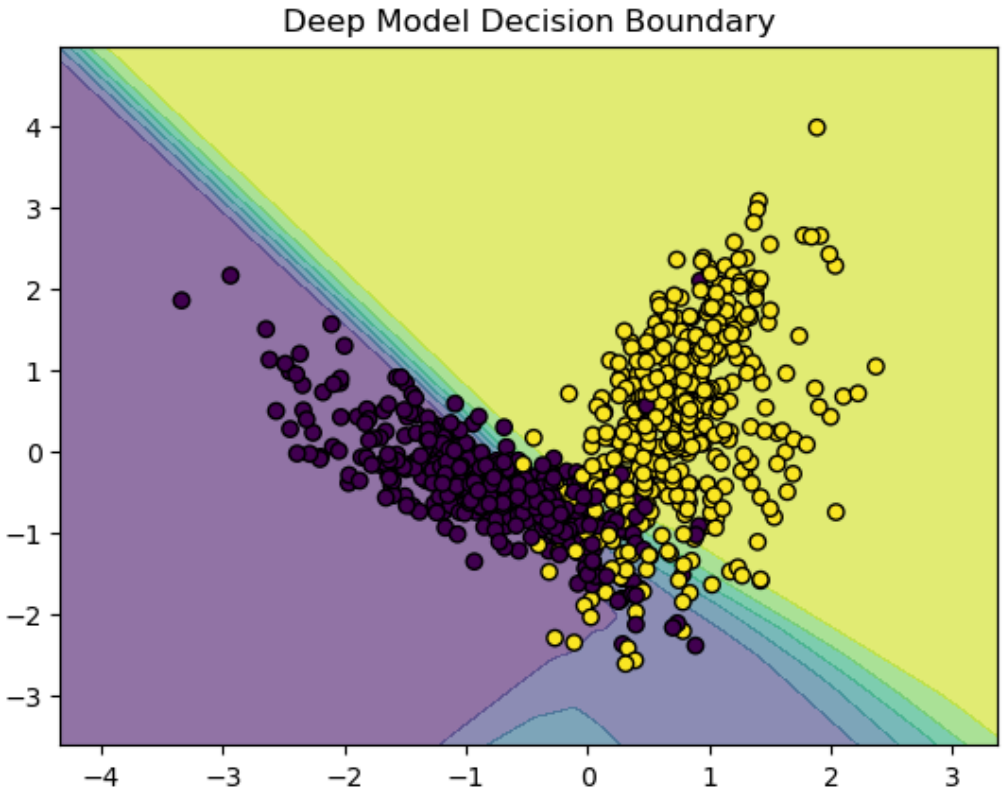
In [10]:
```
plot_decision_boundary(shallow_model, X_train, y_train, "Shallow Model Decision Bou
```

```
1250/1250 [==============================] - 2s 2ms/step
```



In [11]:
```
plot_decision_boundary(deep_model, X_train, y_train, "Deep Model Decision Boundary"
```

```
1250/1250 [==============================] - 2s 2ms/step
```



Report

In this lab, we designed and compared a shallow neural network with one hidden layer and a deep neural network with multiple hidden layers to perform binary classification on a randomly generated dataset with three features. After preprocessing the data through train-test splitting and feature standardization, both models were trained and evaluated using accuracy, precision, recall, and F1-score. The results showed that both models achieved strong performance, with the deep network sometimes converging faster and learning a slightly more flexible decision boundary. However, the shallow model performed nearly as well on the test data, indicating that the problem did not require high model complexity. This exercise demonstrated that while deep networks have greater representational power and can model more complex nonlinear patterns, they also introduce more parameters and potential risk of overfitting. Ultimately, the key insight is that model complexity should align with problem complexity—deeper architectures are powerful but not always necessary for simpler classification tasks.

In [ ]:

```

```